

Mining Taxonomy-aware Colocations: A Summary of Results

Jayant Gupta
gupta423@umn.edu
Computer Science & Engineering
University of Minnesota
USA

Arun Sharma
sharm485@umn.edu
Computer Science & Engineering
University of Minnesota
USA

ABSTRACT

Given a collection of Boolean spatial feature-types, their instances, a neighborhood relation (e.g., proximity), and a hierarchical taxonomy on the feature-types, taxonomy-aware colocation pattern discovery finds the subsets of feature-types or their parents frequently located together. Taxonomy-aware colocations are important due to their use in taxonomy-reliant societal applications in ecology (e.g., finding new symbiotic relationships across food-chain), spatial pathology (e.g., immunotherapy for cancer), etc. Due to the taxonomy, the number of candidate patterns increases considerably (i.e., exponential in the number of collocated instances, where a subset of instances have a parent-child relation). Existing algorithms for mining general colocations are not designed to use taxonomy and will incur redundant computations across the hierarchy. We propose a taxonomy-aware colocation miner (TCM) algorithm which uses a user-defined taxonomy to find taxonomy-aware colocation patterns. We also propose TCM-Prune algorithm that prunes duplicate colocations instances having a parent-child relation. Experiments with synthetic and real data sets show that TCM and TCM-Prune can find colocation patterns missed by the traditional approach (i.e., the ones which do not take hierarchy into account), and TCM-Prune can remove duplicate colocation instances.

CCS CONCEPTS

• Information systems → Data mining; • Computing methodologies → Spatial and physical reasoning.

KEYWORDS

Colocation, Taxonomy-aware pattern mining, Pruning

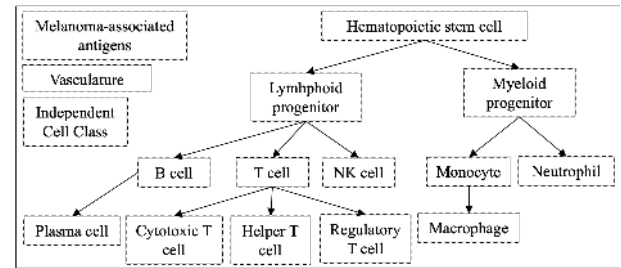
ACM Reference Format:

Jayant Gupta and Arun Sharma. 2022. Mining Taxonomy-aware Colocations: A Summary of Results. In *The 30th International Conference on Advances in Geographic Information Systems (SIGSPATIAL '22)*, November 1–4, 2022, Seattle, WA, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3557915.3561034>

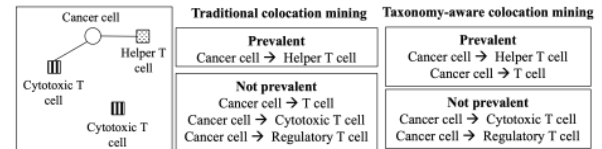
1 INTRODUCTION

Given a set of Boolean spatial feature-types (i.e., their instances satisfy *type_of* property), their instances, a neighborhood relation, and a hierarchical taxonomy on the feature-types we aim to

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SIGSPATIAL '22, November 1–4, 2022, Seattle, WA, USA
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9529-8/22/11.
<https://doi.org/10.1145/3557915.3561034>



(a) Cellular taxonomy of white blood cells [16].



(b) Example of contrast between traditional colocation mining and taxonomy-aware colocation mining. The taxonomy, instances, neighborhood relationship, and parameter thresholds are provided as inputs. The neighborhood graph (show in left) is an intermediate output representing instances which agree to the neighborhood relationship. Taxonomy-aware methods can find {Cancer cell, T cell} as a prevalent colocation candidate missed by traditional colocation methods which do not consider any taxonomy.

Figure 1: Taxonomy-aware colocation mining.

find taxonomy-aware colocation patterns. A colocation is a set of feature-types (e.g., cells, molecules) which are frequently located together [22]. A taxonomy provides a parent-child relationship between different spatial feature-types. A taxonomy-aware colocation therefore is defined as a set of spatial feature-types or their parents in a taxonomy whose instances are frequently located together. Figure 1 shows a taxonomy and an example to contrast between traditional colocation mining and taxonomy-aware colocation mining. As shown, Figure 1(a) is a real-world cellular taxonomy of different white blood cells [16]. The taxonomy provides a rich organization and is helpful for an in-depth analysis of tissue samples for disease progression or to evaluate preventive measure effectiveness (e.g., cancer). Figure 1(b) shows a set of 4 instances of different feature types (i.e., Cytotoxic T cell, Helper T cell, and Cancer cell), where the edge represents a neighborhood relationship. The resulting neighborhood graph is used to mine the colocations. An example of real-world cellular dataset is shown in Figure 9 (Section 6.2).

The traditional colocation mining methods [10, 13, 14, 22, 25, 27] do not consider hierarchies across feature types. This results in *non-consideration* of a feature for a colocation even if its children's feature-types are present or it results in *under-counting* of a given feature-type, since the information on its children's feature-types is unknown. The use of taxonomy can enrich and thus increase the number of colocation candidates and their instances significantly.

Table 1: Application domain and example use-case of taxonomy-aware colocations.

Ecology and environment: Rich taxonomies, such as, the Environment Ontology [7] can be used to mine novel colocations across different organisms. Use of taxonomies is seen as critical to advancing ecological research [18].
Spatial pathology: The taxonomy in Figure 1(a) is a real-world taxonomy which can be used to find taxonomy-aware colocations at the cellular level to help better inform scientists about the interaction between tumor and white blood cells.
Molecular biology: Genetic analysis is critical for developing new medications. Taxonomies such as the Gene Ontology resource [8] can be used to find novel spatial configurations between different genes.
Transportation: Mining spatio-temporal colocations between congested highway sections is important for informed traffic management. The increasing availability of hierarchical point of interest (POI) (e.g., SafeGraph POI dataset [21]), street (e.g., OpenStreetMap taxonomy [20]) datasets makes it possible to find highway sections collocated with particular POIs.
Economics: Knowledge of collocated business categories facilitates investment decisions. It also helps with location searches for opening or establishing new businesses (e.g., a new warehouse, manufacturing unit). Rich taxonomies (e.g., NAICS [6]) can provide additional colocations to address such needs.

A high number of participating instances increases the chance for a candidate colocation to be prevalent (i.e., frequent) as measured by the conditional probability of a feature-type (more details in Section 2.1).

For the given example, a high proximity of cancer cells with T cells or their sub-type is desired. If the taxonomy is not considered the prevalence of {Cancer cell, T cell} is 0, whereas when the taxonomy is considered it is $0.67(\frac{2}{3})$. For other candidate colocations, {Cancer cell, Helper T cell}, {Cancer cell, Cytotoxic T cell}, {Cancer cell, Cytotoxic T cell} the prevalence is the same for both colocation mining techniques, that is, 1, 0.5, and 0 respectively. For the example we assume a conditional probability threshold of 0.6 for a colocation to be prevalent.

The ability to detect taxonomy-aware colocations is becoming critical due to an increased availability of taxonomy-aware datasets in societally important fields such as cancer immunotherapy [16], analysis of Alzheimer’s disease data [15], etc. Besides applications at cellular-scale there are other relevant societal applications which lie in retail analysis, facilities search, etc. In such applications, established business taxonomies (e.g., NAICS [6] with 5 levels of taxonomy) can be used to find taxonomy-aware colocations. Table 1 lists additional application domains such as ecology, spatial pathology, transportation, etc., along with a few example use cases and taxonomies.

There are multiple challenges to this problem. First, there is an exponential number of possible candidates to check for colocations. In addition, the use of taxonomy can further increase the number of colocation candidates and their instances significantly. For example in Figure 1(b), the possible colocation candidates of size 2 without using taxonomy is 6 (i.e., $\binom{4}{2}$) and with, it is 10 (i.e., $\binom{5}{2}$). The real-world NAICS taxonomy has as many as 1056 leaves and

1143 non-leaves spread across 5 levels. The size of such taxonomies makes scalability a challenge. Second, if a colocation candidate has feature-types having a parent-child relationship, then, spurious and duplicate instances can be generated which need to be removed for correctness and completeness. For example, in Figure 1(b) if we consider a colocation candidate (C) of size 2 i.e., $C_2 = \{\text{T-cell, Cytotoxic-T cell}\}$, a single instance (say p_1) of Cytotoxic T cell is counted twice because it is considered the same as a T cell, leading to a C_2 instance of $\{p_1, p_1\}$. Such instances are spurious. Duplicate instances are generated when a colocation has two or more features-types with a parent-child relationship resulting in multiple permutations of an instance. Third, due to the parent-child relationship between spatial features, when the duplicate colocations are removed, the computation of various interest measures (e.g., participation index [14]) becomes non-trivial. Removal of duplicate instances can also lead to undercounting of an instance for a parent-feature resulting in a lower participation index value (Details in Section 4).

Related work: The colocation problem was first proposed by Shekhar [22] as spatial colocation pattern (SCP) mining. The problem was later generalized [14] and has been extensively refined (e.g., [25, 27]). SCP mining has also been explored from the perspective of different data types (e.g., extended spatial objects [10], dynamic datasets [13], etc.) and scale (e.g., [17]). Statistically robust colocation pattern mining is another avenue of research [5]. More details on the development of the field of SCP mining can be found in [24]. Despite being a logical consideration in SCP mining, taxonomy has been largely ignored in the literature. The one study [3] that we found uses taxonomy to refine colocations in a post-processing phase to refine their colocation patterns. Such an approach results in the under-counting of a given feature-type, since the information on its children’s feature-types are not used for candidate generation. This may result in several missed patterns due to low participation index of the parent feature-types in the post-processing phase. However, neither [3] nor general SCP mining can be considered complete if it ignores taxonomy when generating candidate colocations early in the process.

To overcome the limitations of the related work, we propose a taxonomy-aware colocation miner (TCM) algorithm which extends the general colocation mining approach for using taxonomy and then we propose a more refined TCM-Prune algorithm which removes duplicate hierarchical k-colocations (Definition 2.2).

Contributions: Our key contributions are below:

- We formally define taxonomy-aware colocations.
- We propose a taxonomy-aware colocation miner (TCM) method to compute colocations where the spatial feature-types can have parent-child relationships.
- We then propose a pruning based taxonomy-aware colocation miner (TCM-Prune) method. We also provide a procedure for correcting the participation index during the pruning phase.
- We provide a theoretical evaluation of both TCM and TCM-Prune for their computational complexity, correctness, and completeness.
- We provide experimental evaluation using a synthetic dataset to compare an existing colocation miner algorithm with the proposed TCM and TCM-Prune.

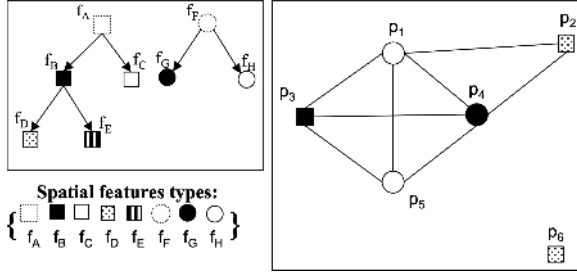


Figure 2: An example of a taxonomy (left) on different spatial feature-types and a neighborhood graph (right).

- We provide a real world case study on tumor data showing that the proposed techniques find interesting and useful patterns.

Scope: We assume a neighborhood relationship is reflexive and that the taxonomy used by TCM-Prune is a set of trees, i.e., each feature-type has single parent at each higher level in the hierarchy. A detailed discussion on the determination of minimum threshold values given as user input (e.g., prevalence, conditional probability) falls outside the scope of this work. This work is limited to instances following *type_of* property of a feature-type and does not consider additional relevant properties such as scale, intensity, etc.

2 BASIC CONCEPTS AND PROBLEM DEFINITION

For keeping the paper self-contained, Section 2.1 provides background knowledge on colocation patterns and its interest measures.

2.1 Basic concepts from colocation literature

An **instance** is a spatial object which is a boolean feature-type (e.g., supermarket, airport at mile-scale and cancer cell, tumor cell at micro-scale), has a spatial location, and can be related to other instances through a spatial **relation** $\mathcal{R}$ (e.g., neighbors). A **k -colocation** (C_k) pattern is a set of k feature-types. An instance of a k -colocation pattern is a set of objects of any distinct feature-type which can form a clique given the input relation and is represented here as a **k -instance**. A **neighborhood graph** is a graphical representation of C_1 instances where the spatial relation between two instances is an edge.

Figure 2 gives an example of each of the concepts. For example, p_1 is an instance of spatial feature-type f_H , the edge between p_1 and p_3 represents a relation between the two instances. Further, $\{p_1, p_3\}$ can form a 2-colocation, if they are prevalent (i.e., frequent or common), and the set of all the edges and instances represents a neighborhood graph. A similar neighborhood graph can be created for the cell based taxonomy described in Figure 1(b).

In colocation pattern mining, participation index (PI) and conditional probability are often used as a measure of the prevalence of a k -colocation pattern. Further, conditional probability serves as a probabilistic interpretation of the colocation candidate pairs. It is mathematically similar to the participation ratio and represents the probability of finding feature(s) f_i given feature(s) f_j .

The participation index is defined in terms of the **participation ratio** (pr) of a feature-type f_i in a colocation pattern C_k . A $pr(C_k, f_i)$

is the fraction of instances of the feature-type participating in k -instances of the pattern. The **participation index** of the pattern, $pi(C_k)$ is the minimal participation ratio of all the feature-types in C_k . For example, consider the colocation pattern of size 2, $C_2 = \{f_D, f_G\}$ in Figure 2. Then $pr(C_2, f_D) = 0.5$ and $pr(C_2, f_G) = 1$ and $pi(C_2) = \min\{pr(C_2, f_D), pr(C_2, f_G)\} = 0.5$. The **prevalence** of a colocation pattern is determined by its participation index. The **conditional probability** of a colocation rule (C_k), $cp(C_j \Rightarrow C_{k-j})$ is the fraction of C_j instances which satisfy the relation $\mathcal{R}$ with C_{k-j} . In Figure 2, the conditional probability $cp(f_D \Rightarrow f_G)$ is 0.5.

A **taxonomy** (τ) of feature-types represents an ‘is_a’ relation between two boolean feature-types (e.g., a Cytotoxic T Cell *is_a* T Cell). Similar to Srikant and Agrawal [23], we model a taxonomy as a directed acyclic graph (DAG) = (F, E) , where F is the set of boolean feature-types and E is the set of ‘is_a’ relationships. If there is an edge $e \in E$ between two spatial feature-types $f, g \in F$, then there is a parent-child relationship between the two. We use $\hat{f}$ to denote the immediate parent of f and $\neg f$ as the set of all peers (i.e., having the same parent) of f without f . Further, we denote all the children of a feature-type f as $ch(f)$. Finally, we assume that a given taxonomy is a set of trees and each feature has a single parent at each higher level in the tree.

Figure 2 shows a taxonomy on different spatial feature-types, where a feature-type f_D ‘is_a’ f_B and f_B can be represented by $\hat{f}_D$ or $\hat{f}_E$. Similarly, f_A can be represented by $\hat{f}_B, \hat{f}_D$, or $\hat{f}_E$. Further, f_D is $\neg f_E$ and vice versa. We note that the taxonomy ‘defined here is similar to the concept of ontology described by Gruber [11] and adopted by [3] for their work. For consistency, we will use the term *taxonomy* in the following text.

2.2 Taxonomy-aware concepts

We extend these concepts to define taxonomy-aware colocation patterns and structures i.e., patterns which occur due to the parent-child relationships amongst the feature-types. We also show how the participation index is computed for colocations when considering a non-leaf spatial feature (i.e., a parent).

DEFINITION 2.1. Given a taxonomy (τ), a **taxonomy-aware k -colocation** is defined as a pattern $C_k = \{f_1, \dots, f_k\}$, which can have colocation candidates containing a parent and one or more of its children.

In Figure 2, $\{f_F, f_A\}$ is a taxonomy-aware 2-colocation. The use of a taxonomy has resulted in the detection of a candidate colocation pattern that might otherwise have been missed.

DEFINITION 2.2. A **k -linear hierarchical path** (LHP_k) is an ordered set of boolean feature-types $\{f_1, \dots, f_k\}$ such that f_i is a parent of f_{i+1} or above in the hierarchy and f_1 is the highest feature-type in the hierarchy (i.e., $\nexists f_i \in LHP_k$ s.t., $f_i = \hat{f}_1$) and f_k is the lowest feature-type in the hierarchy (i.e., $\nexists f_i \in LHP_k$ s.t., $f_k = \hat{f}_i$).

In Figure 2 $\{f_A, f_B, f_D\}$ and $\{f_A, f_B, f_E\}$ are 3-linear hierarchical paths (LHP_3) and $\{f_A, f_B\}$, $\{f_A, f_C\}$, $\{f_A, f_D\}$, $\{f_A, f_E\}$, $\{f_F, f_G\}$, and $\{f_F, f_H\}$ are 2-linear hierarchical paths (LHP_2).

The presence of a k -linear hierarchical path in a colocation candidate can lead to duplicate instances which are a permutation of each other. The duplication occurs between the colocation instances of

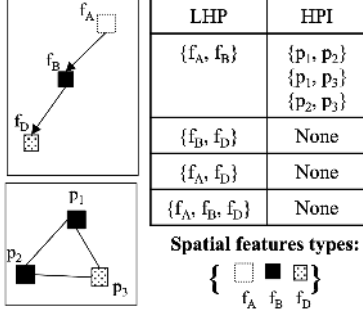


Figure 3: An example illustrating hierarchically permutable k-instance.

the lowest feature-type in the hierarchy. We define such instances as a hierarchically permutable k-instance.

DEFINITION 2.3. A **hierarchically permutable k-instance** (HPI_k) is an instance of a k-colocation where every 1-instance in HPI_k corresponds to a feature-type in a k-linear hierarchical path where each 1-instance in the k-instance belongs to the lowest feature-type in the hierarchy and every pair of 1-instances satisfies the neighborhood relation. Mathematically, for a $P = \{p_1, \dots, p_k\} \in HPI_k$, $\forall p_i, p_j \in P, \mathcal{R}(p_i, p_j)$ is True $\wedge \forall p_i, p_j \in f_k$.

Figure 3 provides an example using a 3 node hierarchy and a set of 3 instance. The 2-colocation instances of $\{p_1, p_2\}$, $\{p_1, p_3\}$, and $\{p_2, p_3\}$ form hierarchically permutable 2-instances for colocation $\{f_A, f_B\} \in LHP_2$. No such instances are observed for $\{f_A, f_D\}$ and $\{f_B, f_D\}$ as p_3 is the only lowest level instance.

No duplication will occur for colocation candidates with peer feature-types (i.e., feature-types with the same parents). For example, in Figure 2, instances p_1 and p_5 form a hierarchically permutable 2-instance for a colocation $\{f_F, f_H\}$. However, p_1 and p_4 do not form such an instance because $p_1 \in f_H$, $p_4 \in f_G$ and $\{f_G, f_H\} \notin LHP_2$.

The presence of hierarchically permutable k-instances increases the participation ratio of the parent feature-types. For example, if we consider a colocation of type $\{f_F, f_H\}$, both p_1 and p_5 contribute towards the participation ratio for f_F (i.e., $\widehat{f_H}$) for a single relation, $\mathcal{R}(p_1, p_5)$. It is because f_F is the parent of f_H that both p_1 and p_5 can be counted as feature-type f_F .

Calculating the participation ratio of a spatial feature-type requires taking into account the boolean spatial feature-type's instances and its children's instances. Thus, for a given feature-type (f) its participation ratio is calculated as follows:

$$pr(C, f) = \frac{Count(C, f) + \sum_{y \in ch(f)} Count(C, y)}{|f| + \sum_{y \in ch(f)} |y|}$$

where, $ch(f)$ denote the set of children of f , $Count(C, f)$ denotes the count of participating instances of type f in colocation C , and $|f|$ denotes the number of instances of feature-type f . For example in Figure 2, $pr(\{f_D, f_E\}, f_E) = \frac{1+1}{1+2} = \frac{2}{3}$.

The participation ratio of a parent with no instances of its own, will be bounded by the highest and the lowest participation ratios of the children. The result can be formalized as follows:

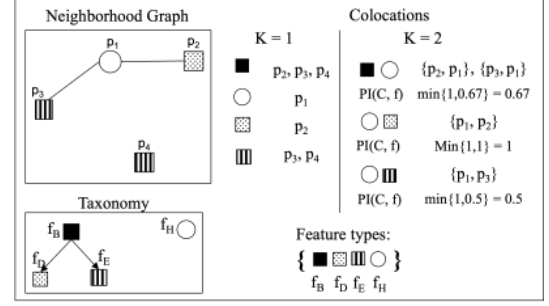


Figure 4: Example showing the behavior of participation index for taxonomy-aware colocation mining.

LEMMA 2.1. If a parent feature-type ($\widehat{f}$) has no instances i.e., $|\widehat{f}| = 0$, then its participation ratio is always bounded by the participation ratio of the most participant child and the least participant child in a colocation C of the form $\{ch(f), G\}$, where $G \wedge ch(f) = \emptyset$ and $G = \{g_1, \dots, g_n\}$.

PROOF. Proof details can be found in Appendix A. $\square$

2.3 Problem Formulation

The problem can be defined formally as follows:

Input:

- A taxonomy $\tau = (F, E)$, where F is a set of M Boolean spatial feature-types $F = \{f_1, f_2, \dots, f_M\}$ and E is the set of 'is_a' relations between the elements of F .
- A set of N instances $P = \{p_1, p_2, \dots, p_N\}$, each $p_i \in P$ is a vector $\langle \text{instance-id, spatial feature-type, location} \rangle$ where spatial feature-type $\in F$ and location in the study area S .
- A neighbor relation $\mathcal{R}$ over location in S .
- A min prevalence threshold value (λ), min conditional probability threshold (θ).

Output: Taxonomy-aware colocation rules having a high prevalence and a high conditional probability.

Objectives: Completeness and Correctness.

Constraints:

- $\mathcal{R}$ is symmetric and reflexive.
- The taxonomy τ is a set of trees.

The use of taxonomy can increase the number of colocation candidates and their instances significantly which may result in unnecessary computations across child and parent feature-types. For example, consider the set of colocation candidates with 2 elements (i.e., C_2) in Figure 4. Traditional algorithms will not consider $\{f_B, f_H\}$ as a candidate pattern as there is no spatial object of type f_B whereas, a taxonomy-aware approach will. The use of taxonomy will result in 4 instances instead of 2. Further, for a colocation of type $\{f_B, f_D\}$, an instance $\{p_2, p_2\}$ is generated, which is unwanted.

Further, if we assume a minimum prevalence threshold (λ) of 0.6, the traditional algorithms will remove $\{f_E, f_H\}$ due to its lower prevalence and the relationship between the instances p_1 and p_2 will be lost. This relationship can be preserved if the colocation of the type $\{f_B, f_H\}$ was considered. Similarly, for larger datasets, many other patterns may be missed. We will use the idea of including children's instances as part of their parent's instances to develop the baseline taxonomy-aware colocation miner as described below.

3 BASELINE APPROACH

Key Idea: The proposed baseline approach modifies the given data by adding children’s instances to their parent’s instances. For example, Figure 4 shows the relationship between $p_1(\in f_H)$ and $p_2(\in f_D, \in f_B)$ can be modeled as a 2-colocation candidate of type $\{f_H, f_D\}$ or $\{f_H, f_B\}$, where f_B is the parent of f_D (i.e., $f_B = \widehat{f_D}$).

The addition of parents can result in spurious instances (e.g., $\{p_2, p_2\}$ as $\{f_B, f_D\}$) of size 2 after the first join of colocation candidates having the form $\{f, \widehat{f}\}$. If allowed, these spurious instances can result in even larger spurious instances of size greater than 2. Below, we first generalize such patterns for $k > 2$ as spurious k -instances and later we will describe the procedure to filter such instances.

DEFINITION 3.1. A *spurious k -instance* is a k -colocation $C_k = \{f_1, \dots, f_k\}$, such that $\exists p_i \in f_i, p_j \in f_j$, where $p_i = p_j \wedge f_i = \text{child}(f_j)$.

With an initial update to the input instances (termed as Instance-Addition algorithm) and the provision of filtering spurious k -instances (termed as C_2 -filtering procedure) the general colocation miner (GCM) algorithm [14] can be used to mine taxonomy-aware colocations. Below, we first describe the Instance-Addition algorithm, then we briefly summarize the steps of the general colocation miner algorithm [14] with the C_2 -filtering procedure.

Instance-Addition (Algorithm 1): The procedure receives two inputs: (1) a flattened taxonomy (τ) which is a hashmap with child as the key and a list of all its ancestors as the value and (2) a hashmap HM_1 with all the existing spatial feature-type as the key and a list of their instances as the value. We initialize a new hashmap $\widehat{HM}_1$ to store the modified HM_1 . The procedure described is composed of two steps with multiple operations as described below.

Step-1: For every feature-type (say f_i) in HM_1 , we access its instances ($HM_1[f_i]$), where ‘[]’ in the algorithm denotes key lookup in a hashmap data structure (Line 5) and the list of its parents (Line 6). Then, for each parent ($\widehat{f_i}$) of the feature-type (f_i) we append the feature-type’s instances to the parent’s instances (Line 7-11). The ‘if-else’ clause is a key initialization check for the hashmap $\widehat{HM}_1$.

Step-2: Step-1 may result in an overlap of instances. For example, an instance of f_i is added twice to $\widehat{f_i}$. Once as a parent of $\widehat{f_i}$ and once as a grandparent of f_i . We remove duplicate instances of each feature-type (Line 13-14) by using the ‘set’ operation on the instance list which removes duplicate elements in a list.

Taxonomy-aware Colocation Miner (Algorithm 2): The baseline algorithm takes a hierarchical taxonomy τ representing *is_a* relation between feature-types in F (a set of M boolean spatial feature-types), a set P of feature-type instances, a user-defined function for spatial neighborhood relationships ($\mathcal{R}$), and a threshold prevalence (λ) and conditional probability (θ).

First, the data structures for size 1 colocations are initialized (Line 2). The colocations of size 1 are prevalent (since all instances are participating) and hence form the candidate patterns for larger (i.e., $k > 1$) k -colocations. Then, we generate a hashmap of instances (Line 3) used by the Instance-Addition procedure (Line 4) to update candidate 1-colocations (C_1) which is used to initialize T_1 (Line 5).

The baseline algorithm for mining colocation rules iteratively performs four basic tasks: (1) generation of candidate colocations, (2)

Algorithm 1 Instance-Addition

Input: Flattened taxonomy τ, HM_1
Output: $\widehat{HM}_1$

```

1: procedure INSTANCE-ADDITION
2:    $\widehat{HM}_1 \leftarrow \{\}$ 
3:   Step 1: Add children’s instances to parent’s instances.
4:   for each:  $f_i \in HM_1$  do
5:      $Instances(f_i) \leftarrow HM_1[f_i]$ 
6:      $List(\widehat{f_i}) \leftarrow \tau[f_i]$ 
7:     for each:  $\widehat{f_i} \in List(\widehat{f_i})$  do
8:       if  $\exists \widehat{f_i} \in \widehat{HM}_1$  then
9:          $\widehat{HM}_1[\widehat{f_i}] \leftarrow \widehat{HM}_1[\widehat{f_i}] \cup Instances(f_i)$ 
10:      else
11:         $\widehat{HM}_1[\widehat{f_i}] \leftarrow Instances(f_i)$ 
12:   Step 2: Remove duplicate instances.
13:   for each:  $f_i \in \widehat{C}_1$  do
14:      $\widehat{HM}_1[f_i] \leftarrow set(\widehat{HM}_1[f_i])$ 
```

Algorithm 2 Taxonomy-aware Colocation Miner

Input: $\tau, P, \mathcal{R}, \lambda, \theta$
Output: Colocations (C) with $pi(C) > \lambda$ and $cp(C) > \theta$.
Variables:

- k : colocation size
- C_k : set of candidate k -colocations
- T_k : set of table instances of colocations in C_k
- P_k : set of prevalent k -colocations
- R_k : set of colocation rules of size k

```

1: procedure TAXONOMY-AWARE COLOCATION MINER:
2:    $k \leftarrow 1; C_1 \leftarrow P; P_1 \leftarrow P;$ 
3:    $HM_1 \leftarrow generate\_hashmap(P)$ 
4:    $\widehat{HM}_1 \leftarrow \text{Instance-Addition}(\tau, HM_1)$ 
5:    $T_1 \leftarrow initialize\_table\_instance(\widehat{HM}_1)$ 
6:   Initialization:
7:   for each:  $k : 2 \Rightarrow K$  do
8:      $C_k, T_k, P_k, R_k \leftarrow \phi;$ 
9:   while (not empty  $P_k$  and  $k < K$ ) do
10:     $C_{k+1} \leftarrow generate\_candidate\_colocation(C_k, k);$ 
11:     $T_{k+1} \leftarrow generate\_table\_instance(\lambda, C_{k+1}, T_k, \mathcal{R});$ 
12:    if  $k = 1$  then
13:       $T_{k+1} \leftarrow C_2\_Filtering(T_{k+1})$ 
14:     $P_{k+1} \leftarrow select\_prevalent\_colocation(\lambda, C_{k+1}, T_{k+1});$ 
15:     $R_{k+1} \leftarrow generate\_colocation\_rule(\theta, P_{k+1}, T_{k+1});$ 
16:     $k \leftarrow k + 1;$ 
17:   return  $R_2 \cup R_3 \cup \dots \cup R_{k+1};$ 
```

generation of table instances of candidate colocations, (3) pruning based on prevalence, and (4) generation of colocation rules. These tasks are carried out inside a loop over the size (k) of the colocations. The loop starts from $k = 2$. Below, we summarize these tasks.

1. **Generation of candidate colocations:** An *apriori_gen* [2] algorithm is used to generate size $k + 1$ candidate colocations from size k prevalent colocations. The function takes the set of all prevalent size k colocations (P_k). Then, k -colocations with $k - 1$ similar feature-types are joined. Following this, all colocations $c \in C_{k+1}$ such that some size k subset of c which is not in P_k are pruned. The function outputs a list of instance_id pairs where each pair corresponds to the table to be joined. (Line 10)

2. **Generation of table instances of candidate colocations:** The pair of table instances of C_k are joined. The join has two constraints: (1) a combinatorial constraint, which specifies that similar $k - 1$ feature-types have the same instances and (2) a spatial constraint whereby non-similar feature-types must satisfy the neighborhood relationship. (Line 11)

As mentioned earlier the first join will result in spurious 2-instances. To avoid spurious k -colocations, we use C_2 -Filtering (Line 12-13) which prunes spurious 2-instances after the first join. For a given feature-type f the filter removes $|f|$ spurious 2-instances. The step also ensures that no spurious k -instances are generated for $k > 2$. Below we prove this formally.

LEMMA 3.1. *C_2 -filtering guarantees that no spurious k -instances are generated for $k > 2$.*

PROOF. Assume spurious 2-instances are filtered. Then consider a 3-colocation candidate $C_3 = \{f_i, f_j, f_k\}$. The candidate can be generated in the following three ways: (1) $\{f_i, f_j\} \bowtie \{f_j, f_k\}$, (2) $\{f_i, f_k\} \bowtie \{f_j, f_k\}$, and (3) $\{f_i, f_j\} \bowtie \{f_i, f_k\}$, where $\bowtie$ is an inner join operator. For the cases (1) and (2) no spurious 3-instances are generated since spurious 2-instances have already been filtered out. For the third case (3) the join checks for the spatial relationship between f_j and f_k ($\mathcal{R}(f_j, f_k)$). Since the relation is defined by the set of 2-colocations instances of $\{f_j, f_k\}$, which has no spurious 2-instance, the join results in no spurious 3-instances. Similarly, the proof can be generalized for $k > 3$. $\square$

3. Prevalence based pruning: Prevalent colocations are selected by comparing the participation index to the prevalence threshold λ . This step can be optimized using sets to check unique instances for each feature-type. We note that the count is based on updated $\widehat{HM}_1$ (Line 14).

4. Generation of colocation rules: Colocation rules which are above the user defined conditional probability threshold (θ) are selected. The function can be efficiently implemented using a hashmap. (Line 15)

LEMMA 3.2. *TCM is complete.*

PROOF. General colocation miner (GCM) can miss taxonomy-aware colocations as it does not consider an instance as its parent feature-type and beyond. The Instance-Addition procedure adds the instances of a feature-type to its parent feature-types resolving the issue. Rest of the proof follows from the result that GCM is complete [14]. $\square$

LEMMA 3.3. *TCM is correct.*

PROOF. The addition of an instance to its feature-supertype results in its duplication resulting in spurious patterns. The C_2 -filtering procedure removes spurious 2-colocation instances which guarantees that no spurious k -colocation instances are generated. Further, the correctness of GCM [14] guarantees that all the k -colocation instances satisfy the neighborhood relation. $\square$

Execution Trace: Figure 5 shows the execution trace of GCM, TCM, and TCM-Prune (described later). As shown there are four spatial feature-types, three of which form a 3-linear hierarchical path (i.e., f_A, f_B, f_D) and one single feature-type f_H . We assume both prevalence threshold (λ) and conditional probability threshold (θ) to be 0.75. Further, instances pruned in TCM and TCM-Prune are marked as solid red and dashed brown boxes respectively.

As shown, the GCM algorithm generates one 2-colocation candidate (shown in the top-right bounded by the dotted box) having 3 instances. The candidate is also prevalent with $pi(\{f_D, f_H\}) = 1$.

The TCM algorithm generates six taxonomy-aware 2-colocation candidates, four taxonomy-aware 3-colocation candidates and one

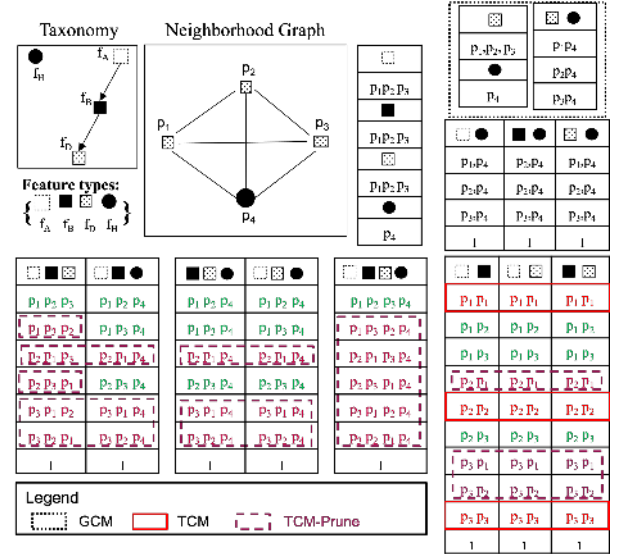


Figure 5: Execution trace of GCM, TCM, TCM-Prune.

taxonomy-aware 4-colocation candidate. Instances bound by the solid red box (e.g., p_1p_1) are spurious colocation candidates, which are pruned using the C_2 -Filtering procedure. There are 27, 24, and 6 instances of size 2, 3, and 4 colocations and all the candidates are prevalent having a participation index of 1.

However, there are several duplicate k -instances (e.g., $\{p_1, p_2\}$ and $\{p_2, p_1\}$). Below we describe the TCM-Prune algorithm, which removes such duplicate patterns and reduces the number of generated instances significantly.

4 PRUNING BASED APPROACH

Key Idea: The baseline TCM approach can be optimized by pruning duplicate hierarchically permutable k -instances (HPI_k), since each unique $P \in HPI_k$ generates $k!$ duplicate instances of the type $C_{k+n} = f_1, \dots, f_k, g_1, \dots, g_n$ (Lemma 4.1).

LEMMA 4.1. *If n unique hierarchically permutable k -instances (HPI_k) are present then $n * k!$ duplicate taxonomy-aware instances of the type $C_{k+n} = \{f_1, \dots, f_k, G\}$ are generated, where $G = \{g_1, \dots, g_n\}$.*

PROOF. Let $P = \{p_1, p_2, \dots, p_k\} \in HPI_k$. Then, all the permutations of P will also be in HPI_k . For each permutation of P a C_{k+n} instance can be generated resulting in $k!$ duplicates. Thus, for n hierarchically permutable k -instances, $(n * k!)$ duplicates are generated. $\square$

We note that pruning is not trivial as it may affect the calculation of the participation ratio of a feature-type in a colocation. For example, consider the colocation $\{f_A, f_B\}$ in Figure 6 which is based on the example in Figure 5. If we remove the repeating HPI_2 instances i.e., (p_2, p_1) , (p_3, p_1) , and (p_3, p_2) , then we get $pr(\{f_A, f_B\}, f_A) = \frac{2}{3}$, $pr(\{f_A, f_B\}, f_B) = \frac{2}{3}$, and $pi(\{f_A, f_B\}) = \frac{2}{3}$. However, the actual participation ratio (and the participation index) is 1. This occurs because the pruned instances p_1 and p_2 do not participate (i.e., not counted towards participation ratio) for the features f_A and f_B respectively. Thus, we propose a HPI -Prune procedure which can remove duplicate HPI_k instances without affecting the participation ratio measures.

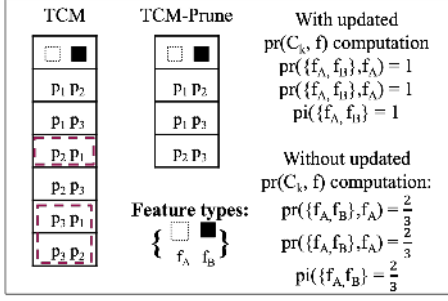


Figure 6: Example showing errors in TCM-Prune without the updated participation index computation.

The procedure receives three inputs a candidate k -colocation (C_k), table instances for the candidate colocation, and the taxonomy (τ). The procedure outputs a filtered table of instances without duplicates and stores the identified LHP_k and HPI_k in a global variable to be used for participation index calculation. Details are as follows:

HPI-Prune (Algorithm 3) : This procedure first verifies if the candidate k -colocation is LHP_k then, which of its instances are HPI_k .

For the first we check if there is a parent-child relation between every two feature-types using the taxonomy. The parent-child relation need not be immediate. If it fails for any pair, we return the input as is, since no pruning is required (Line 2-3). Else, update the global hashmap with the new LHP_k candidate (Line 4).

Before checking for HPI_k , we initialize a variable to store HPI_k instances (Line 5). To find HPI_k , we check if all the objects belong to the lowest hierarchical feature-type in HPI_k (Definition 2.3). Thus, we start by getting the lowest feature-type in LHP_k i.e., f_k (Line 6). Then, for each instance we check if every object (i.e., p_i) is f_k . If not, the instance is not considered further for deletion (Line 9-11).

If an HPI_k is found its ID is stored for deletion and its ordered permutation is stored (Line 12-18). The *reorder()* function is used to sort an instances in a lexicographic order. This allows all the duplicate instances to be represented by a single instance and can be checked in constant time using a hashmap structure. The deletion consists of two steps:

- (1) Delete all the marked instances and replace them with a single instance that is sorted in lexicographic order (Line 19-20). The use of ordering helps to preserve the same instance across tables.

- (2) Update global hashmap with new HPI_k instances (Line 21).

Prevalence based pruning: To compute the participation index of a size k -colocation correctly, the procedure uses (1) a set of known k -linear hierarchical paths (LHP_k) and (2) a set of known hierarchically permutable k -instances (HPI_k). Each set is stored as a global hashmap data structure.

Measuring prevalence: For a given colocation candidate of size k we check if it is LHP_k (using the global data structure). If an LHP_k is found, each of its k -instance is checked for HPI_k . If an HPI_k is found, we count all the single instances as ‘participating’ for each of the feature-types.

For example in Figure 6, first the candidate $\{f_A, f_B\}$ is verified as LHP_2 and then, when the instance $\{p_1, p_2\}$ is verified as HPI_2 both p_1 and p_2 are added as participating instances for f_A and f_B . Similarly, at a later check p_2 will be added as participating

Algorithm 3 HPI-Prune

Input: C_k, T_k, τ
Output: T_k with no duplicate instances
Variables:

- f_k : Stores the lowest feature in hierarchy for a HPI_k
- C_k -Map: Hashmap to store instance-ids to be deleted
- *check_instance*: Boolean for relevant HPI_k instance

```

1: procedure HPI-PRUNE:
2:   if Check-LHP $_k(\tau, C_k)$  is False then
3:     return  $T_k$ 
4:   update_LHP $_k$ _hashmap( $C_k$ )
5:   Initialize  $T_k$ -Map to empty hashmap.
6:    $f_k \leftarrow$  get_lowest_feature( $C_k, \tau$ )
7:   for instance  $\in T_k$  do
8:     check_instance  $\leftarrow$  True
9:     for  $p_i \in$  instance do
10:      if  $p_i \neq f_k$  then
11:        check_instance  $\leftarrow$  False
12:     if check_instance is True then
13:       ordered_instance  $\leftarrow$  reorder(instance)
14:       if ordered_instance  $\in C_k$ -Map then
15:         instance-id  $\cup C_k$ -Map[ordered_instance]
16:       else
17:         Initialize  $C_k$ -Map[ordered_instance]
18:         instance-id  $\cup C_k$ -Map[ordered_instance]
19:   for instance  $\in C_k$ -Map do
20:     Delete( $T_k$ , List  $C_k$ -Map[instance])
21:     update_HPI $_k$ _hashmap(instance)
22:   return  $T_k$ 

```

instances. The addition of p_2 results in a participation ratio of 1, thus, correcting the error in prevalence computation. Below, TCM-Prune snippet (Algorithm 4) shows the placement of HC-Prune and the updated prevalence (i.e., *prevalent_pruned_colocation*) based pruning procedure within TCM.

Algorithm 4 TCM-Prune Snippet

```

1: procedure TCM-PRUNE:
2:   :
3:   :
4:   for candidate  $\in C_{k+1}$ , table  $\in T_{k+1}$  do
5:     table  $\leftarrow$  HC-Prune(candidate, table,  $\tau$ )
6:      $P_{k+1} =$  prevalent_pruned_colocation( $\lambda, C_{k+1}, T_{k+1}$ );
7:   :

```

LEMMA 4.2. *TCM-Prune is complete.*

PROOF. TCM-Prune removes duplicate instances due to hierarchically permutable k -instances (Lemma 4.1). Further, selecting prevalent k -colocations using the pruned instances task ensures that the participation ratio of the pruned colocations is not incorrectly computed. Thus, a colocation instance generated by TCM is generated in TCM-Prune atleast once. Rest of the proof follows from the result that TCM is complete (Lemma 3.2). $\square$

LEMMA 4.3. *TCM-Prune is correct.*

PROOF. The proof follows from Lemma 3.3. $\square$

5 THEORETICAL ANALYSIS

This section examines the time complexity of the TCM and TCM-Prune algorithms and compares it with the GCM algorithm. We assume $|C_k|$ is the number of k -colocation candidates, N_k is the number of k -instances, M is the number of feature types, and k is the colocation size.

5.1 Taxonomy-aware colocation miner

- The initialization step (Line 2) and the generate hashmap step (Line 3) have a cost of $O(N_1)$.
- The Instance-Addition procedure has two steps (Line 4). The complexity of the first step depends on the taxonomy structure (e.g., skewed tree vs balanced tree) and the distribution of instances over feature-types. The best case occurs when all the instances belong to root nodes where the complexity is $O(N_1)$. When the taxonomy is balanced and the instances are uniformly distributed, the complexity is $O(\log(M)) \cdot N_1$. The worst case occurs when the taxonomy consists of skewed trees and the instances are leaf feature-types having a complexity of $O(M \cdot N_1)$.
- Step 2 removes the duplicate instances for each feature-type. It depends on the number of added instances. Let us assume that the number of instances after the first step is $\tilde{N}_1$, where $O(N_1) \leq O(\tilde{N}_1) \leq O(M \cdot N_1)$. Hence, the worst case complexity of the procedure is $O(M \cdot N_1)$.
- Initialization of the table instances is $O(\tilde{N}_1)$ (Line 5) and the initialization of the data structures costs $O(k)$ (Line 6-8).
- Candidate colocations and table instances (Line 10-11) of size k are generated using a nest-loop join strategy with an asymptotic cost of $|C_k|^2 \cdot \delta^2$, where δ is the average number of k -instances for a k -colocation candidate. For $k = 2$, the asymptotic complexity can be simplified to $O(\tilde{N}_1^2)$.
- The C_2 -filtering (Line 12-13) is done once for size 2 colocations. Its worst case cost is $O(\tilde{N}_1^2)$.
- Selection of prevalent colocations (Line 14) can be done in $O(|C_k| \cdot \delta)$, where the table instances can be used to compute the participation index and compare it with the threshold.
- The prevalent colocations (Line 15) can be used to generate colocation rules bounded by $O(|C_k| \cdot \delta)$.

5.2 Pruning based approach

The pruning based approach uses (1) HPI-Prune to remove duplicate colocations and (2) an updated prevalence based pruning step for correct participation ratio calculation. Below we describe its cost:

- The checking of k -linear hierarchical paths costs $O(k^2)$ using nested loops over each k -instances (Line 2-3).
- Getting the lowest hierarchical feature-type is done in $O(k^2)$ using a nested loop over the k feature-types (Line 6).
- The loop (Line 7-18) is done for each instance and the costliest operation is reorder (Line 13), which $O(k \cdot \log(k))$. Thus, the overall cost of the loop operation is $O(|C_k| \cdot \delta \cdot k \cdot \log(k))$.
- The deletion step removes duplicate instances and can be done in $O(|C_k| \cdot \delta)$ assuming linked lists.

The updated prevalence based pruning step adds dictionary lookup operations and a loop with complexity $O(k)$ with the asymptotic cost of $O(|C_k| \cdot \delta)$ (Assuming, $k \ll |C_k| \cdot \delta$).

The additional procedures in TCM-Prune can cost an additional $O(|C_k| \cdot \delta \cdot k \cdot \log(k))$ in the worst case. However, depending on the number of HPI_k instances in the dataset, the additional steps can reduce the cost significantly.

For a k -colocation, let m_i denote the number of LHP_i candidates, where i denotes the size of the subset of feature-types in

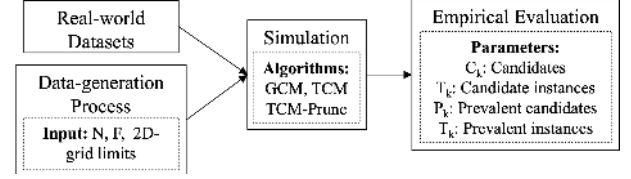


Figure 7: Experiment Design.

a k -colocation forming the LHP. Also, let n_i^k denote the number of HPI_i instances for the LHP_i . Then, TCM-Prune will have $\sum_{i=1}^k (i! - 1) * n_i * m_i$ less instances than TCM for a colocation of size k . Consider Figure 5 for an example. (1) TCM-Prune reduces the size of C_2 instances from 27 to 18 (as, $(2! - 1) * 3 * 3 = 9$), (2) TCM-Prune reduces the size of C_3 instances from 24 to 10 (as, $(3! - 1) * 1 * 1 + (2! - 1) * 3 * 3 = 14$), and (3) TCM-Prune reduces the size of C_4 instances from 6 to 1 (as, $(3! - 1) * 1 * 1 = 5$).

Overall, the complexity of GCM, TCM, and TCM-Prune depend on both the number of colocation candidates and their instances. Compared to GCM, both TCM and TCM-Prune are costly due to a high δ value. Between TCM and TCM-Prune, if the additional cost of pruning i.e., $O(|C_k| \cdot \delta \cdot k \cdot \log(k))$ significantly reduces duplicate candidates, i.e., $\sum_{i=1}^k (i! - 1) * n_i * m_i$, then TCM-Prune will perform better than TCM.

6 EXPERIMENTAL EVALUATION

The goal of the experiments is two-fold: (1) To empirically evaluate the effect of using taxonomy-aware approaches on the number of candidate and prevalent k -colocation patterns detected in a colocation mining task. (2) To evaluate the effect of the proposed pruning-based approach on the number of candidate and prevalent k -colocation instances.

Experiment Design: As shown in Figure 7, the experiments are composed of three key steps. The first step is data generation which is done either using real-world datasets or through a data-generation process. The second step is colocation mining, where we run the three algorithms, namely, the generalized colocation miner (GCM), the taxonomy-aware colocation miner (TCM), and the pruning based taxonomy-aware colocation miner (TCM-Prune). The third step is the empirical evaluation, where we compare the number of candidates and their instances and the number of prevalent candidates and their instances detected by each method.

Choice of threshold values: Values for the min prevalence threshold and the min conditional probability threshold are input by the users. The values depend on the desired size of the results and other user objectives. Although a detailed discussion is outside the scope of this work, there are ways to determine minimum threshold values. For example, if the distribution of an interest measure is known, then top- k [26] or top- $k\%$ co-locations can be determined.

Resources: The algorithms were implemented using python based libraries [1]. The experiments were performed on a system with a 2.2 GHz Quad-Core Intel Core i7 processor and 16 GB 600 MHz DDR3 memory.

6.1 Synthetic Dataset

Data generation: We used a primary data generation procedure which takes 3 inputs: (1) The number of instances to generate (N), (2) A set of spatial feature-types to choose from, (3) 2-dimensional

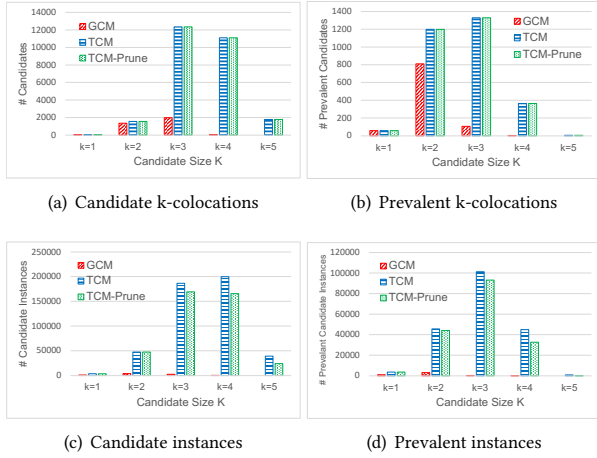


Figure 8: Results on synthetic data

limits ($[l_1, l_2]$) on the study are S , within which the instances must lie. We also define a taxonomy on the set of spatial feature-types.

To create an instance, the module randomly selects its feature-type and a position (x, y) within the grid limits. For our experiments, we used $N = 1000$, a set of 57 feature-types, and limits of $[0, 50]$ for both axes of the study area. For the taxonomy, we used the 57 feature-types to generate 5 full trees of depth 1 to 5 (i.e., having 1 to 31 features).

We used the synthetic data to generate k -colocations using GCM and taxonomy-aware k -colocations using TCM and TCM-Prune. Further, we kept $\lambda = 0.1$, $\theta = 0.5$, and $\mathcal{R}$ as $distance(f_1, f_2) < 2.3$.

Results: Figure 8(a) and Figure 8(b) show that the taxonomy-aware methods generate a significantly larger number of candidates as well as prevalent candidates of k -colocations. The results show the utility of a taxonomy-aware approach to generate patterns sensitive to the taxonomy.

We also note that both TCM and TCM-Prune have the same number of candidates and prevalent candidates. This is because TCM-Prune only reduces duplicate instances, which does not affect the generation of candidates and prevalent candidates.

Figure 8(c) and Figure 8(d) show that TCM-Prune is able to prune duplicate instances (e.g., 33, 923 for candidate 4-colocation instances and 12, 236 for prevalent 4-colocation instances).

6.2 Spatial Pathology by Mapping Cancer cells

Background and Data Description: We considered a tumor dataset [19] which is based on multiplexed immuno-fluorescence (MxIF) images generated via visual inspection of a sample tissue sliced into thin sections to describe certain features (e.g., cell type). A MxIF is a 2 dimensional map of cancer and immune cell center-points specified by coordinates x and y . It can be viewed as an image consisting of multiple categorical points in a Euclidean space where each point is assigned with one or more phenotype markers (i.e., types of tumor cells). Such cellular arrangement in 2D space permits the interplay of proximity relationships among tumor cell types, which can be considered an essential factor in disease progression assessment and treatment [4, 9, 16]. Spatially this dataset is not all that different from POI datasets which provide latitude and longitude or street addresses for various points of interest. These

datasets are societally important for their use in designing cancer immunotherapy.

The dataset had 4692 instances and 18 feature-types. In the taxonomy, there was one tree with 13 feature-types and the largest depth of 4 for 5 feature-types. For this dataset, we kept $\lambda = 0.1$, $\theta = 0.5$, and $\mathcal{R}$ as $distance(f_1, f_2) < 50$. A subset of the dataset is shown in Figure 9.

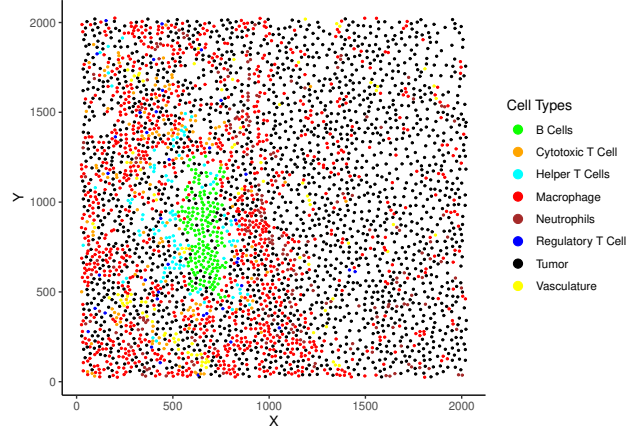


Figure 9: MxIF point patterns detailing spatial arrangements between different cell types. (Best in color)

Results: Figure 10 shows the candidate and prevalent k -colocations and candidate and prevalent instances generated for the k -colocations. Again, TCM and TCM-Prune generates a significantly larger number of candidate k -colocations and k -instances compared to GCM

In addition, the results from this dataset show that depending on the data, TCM-Prune can prune instances with high efficiency. For example, it was able to prune 185, 534 instances of candidate 4-colocations and 162, 517 instances of prevalent 4-colocations. These results further reinforce the usefulness of TCM-Prune to achieve computational efficiency.

Pruning efficiency of TCM-Prune and number of hierarchically permutable k -instance (HPI_k): Due to Lemma 4.1, the number of hierarchically permutable k -instances affects the pruning performance, i.e., a higher number of HPI_k improves the pruning. Indeed, the HPI_k distribution for the synthetic data was just $HPI_2 = 655$, $HPI_3 = 31$ while for the tumor data it was, $HPI_2 = 4306$, $HPI_3 = 3541$, and $HPI_4 = 772$. The higher HPI_k number explains why pruning efficiency for the tumor data was distinctly higher than for the synthetic data.

Run-time analysis: Figure 11 shows the run-time (in seconds) of processing colocations from size 1 to 4 for the two case-studies. We observe the following: (1) The runtime of experiments for Synthetic data is higher than for the Tumor data (e.g., 20-40x more). This is due to a high number of candidate colocations in the synthetic data. (2) The run-time of GCM is significantly low compared to TCM and TCM-Prune algorithms. This is expected as both the number of colocation candidates and its instances are low without taxonomy-awareness. (3) For synthetic data, TCM-Prune is always better than TCM, whereas for the Tumor data TCM-Prune improves progressively finally performing better when $k = 4$. Thus, if the overall

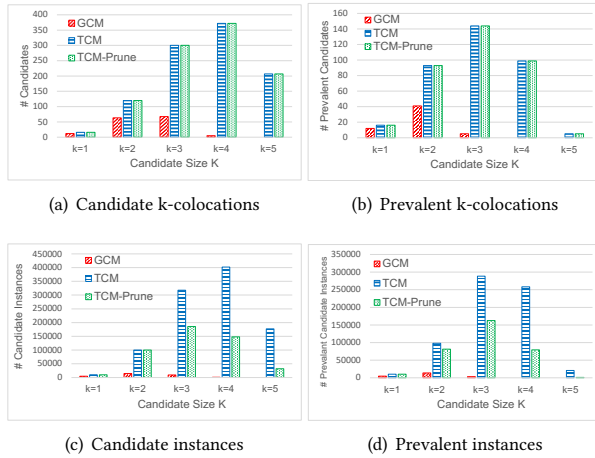
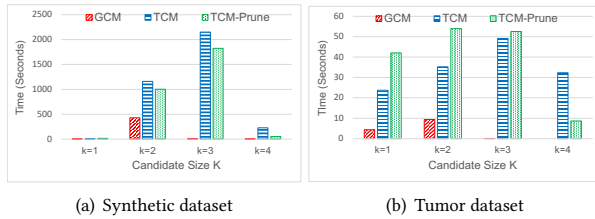


Figure 10: Results on Tumor data

Figure 11: Time cost for different k -colocations.

cost is low then, pruning may not be optimal for lower k , however, with a gradual increase in k TCM-Prune starts to become optimal.

7 CONCLUSION AND FUTURE WORK

In this paper, we formally define taxonomy-aware colocations. Then, we propose a taxonomy-aware colocation miner (TCM) method to compute colocations where the spatial feature-types can have parent-child relationships. We then propose a pruning based taxonomy-aware colocation miner (TCM-Prune) method. We also provide a procedure for the correct calculation of the participation index for the pruned instances. Then, we provide formal results for their correctness, completeness. Finally, we compare the general colocation miner algorithm (GCM) with the proposed TCM and TCM-Prune on synthetic and real-world tumor datasets. Experiment evaluation shows that TCM and TCM-Prune can find colocation patterns missed by the general approach, and TCM-Prune is optimal than TCM.

In the future we will generalize the TCM-Prune algorithm for non-tree taxonomies where a feature-type can have multiple parents. For better computational efficiency, we will investigate taxonomy-based bounds on the k -colocation candidates to limit the candidate generation. Then, we will formalize the pruning efficiency and compare its sensitivity to spatial auto-correlation. Similarly, other we plan to explore location-dependent parameters to account for spatial variability[12]. Finally, we plan to study the effect of different input parameters (e.g., prevalence threshold value) on the output and assess the effectiveness of the algorithms for different neighborhood relations.

ACKNOWLEDGEMENT

This material is based upon work supported by the National Science Foundation under Grants No. 2118285, 1737633, 1901099, and 1916518. We thank Kim Koffolt and the Spatial Computing Research Group for valuable comments and refinements. We also thank the anonymous reviewers for their insightful comments to further refine the paper.

REFERENCES

- [1] 2022. Taxonomy-aware colocation mining. https://anonymous.4open.science/r/generalized_colocation-F105/
- [2] R. Agrawal, R. Srikant, et al. 1994. Fast algorithms for mining association rules. In *Proc. 20th int. conf. very large data bases, VLDB*, Vol. 1215. Citeseer, 487–499.
- [3] Xuguang Bao and Lizhen Wang. 2017. Discovering interesting co-location patterns interactively using ontologies. In *International Conference on Database Systems for Advanced Applications*. Springer, 75–89.
- [4] Souptik Barua, Penny Fang, et al. 2018. Spatial interaction of tumor cells and regulatory T cells correlates with survival in non-small cell lung cancer. *Lung Cancer* 117 (2018), 73–79.
- [5] Sajib Barua and Jörg Sander. 2013. Mining statistically significant co-location and segregation patterns. *IEEE TKDE* 26, 5 (2013), 1185–1199.
- [6] U.S. Census Bureau. 2021. NAICS Codes. <https://bit.ly/3BD62NB>.
- [7] Pier Luigi Buttigieg, Norman Morrison, et al. 2013. The environment ontology: contextualising biological and biomedical entities. *Journal of biomedical semantics* 4, 1 (2013), 1–9.
- [8] Gene Ontology Consortium. 2019. The gene ontology resource: 20 years and still GOing strong. *Nucleic acids research* 47, D1 (2019), D330–D338.
- [9] Majid Farhadloo, Carl Molnar, et al. 2022. SAMCNet: Towards a Spatially Explainable AI Approach for Classifying MxIF Oncology Data. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2860–2870.
- [10] Yong Ge, Zijun Yao, and Huayu Li. 2019. Computing co-location patterns in spatial data with extended objects: a scalable buffer-based approach. *IEEE TKDE* (2019).
- [11] Thomas R Gruber. 1993. A translation approach to portable ontology specifications. *Knowledge acquisition* 5, 2 (1993), 199–220.
- [12] Jayant Gupta, Carl Molnar, et al. 2021. Spatial Variability Aware Deep Neural Networks (SVANN): A General Approach. *ACM Transactions on Intelligent Systems and Technology (TIST)* 12, 6 (2021), 1–21.
- [13] Xin Hu, Guoyin Wang, and Jiangli Duan. 2020. Mining maximal dynamic spatial colocation patterns. *IEEE transactions on neural networks and learning systems* 32, 3 (2020), 1026–1036.
- [14] Y. Huang, S. Shekhar, and H. Xiong. 2004. Discovering colocation patterns from spatial data sets: a general approach. *IEEE TKDE* 16, 12 (2004), 1472–1485.
- [15] Barbara Kramarz and Ruth C Lovering. 2019. Gene ontology: A resource for analysis and interpretation of Alzheimer’s disease data. *Exon Publications* (2019), 23–36.
- [16] Y. Li, M. Farhadloo, et al. 2021. SRNet: A spatial-relationship aware point-set classification method for multiplexed pathology images. In *Proceedings of DeepSpatial’21: 2nd ACM SIGKDD Workshop on Deep Learning for Spatiotemporal Data, Applications, and Systems*, Vol. 10.
- [17] Y. Li and S. Shekhar. 2018. Local co-location pattern detection: a summary of results. In *10th International Conference on Geographic Information Science (GIScience 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- [18] Joshua S Madin, Shawn Bowers, et al. 2008. Advancing ecological research with ontologies. *Trends in ecology & evolution* 23, 3 (2008), 159–168.
- [19] Rachel LG Maus, Alexey A Leontovich, et al. 2022. Resolving the Heterogeneous Tumor-Centric Cellular Neighborhood through Multiplexed, Spatial Paracrine Interactions in the Setting of Immune Checkpoint Blockade. *Cancer Research Communications* 2, 2 (2022), 78–89.
- [20] Openstreetmap.org. 2021. Open street map: United States/Road classification.
- [21] SafeGraph. 2021. Point of Interest (POI) Data.
- [22] S. Shekhar and Y. Huang. 2001. Discovering spatial co-location patterns: A summary of results. In *International symposium on spatial and temporal databases*. Springer, 236–256.
- [23] R. Srikant and R. Agrawal. 1995. Mining generalized association rules.
- [24] Lizhen Wang, Yuan Fang, and Lihua Zhou. 2022. Preference-based Spatial Colocation Pattern Mining. *Big Data* (2022).
- [25] Xiaojing Yao, Ling Peng, et al. 2016. A fast space-saving algorithm for maximal co-location pattern mining. *Expert Systems with Applications* 63 (2016), 310–323.
- [26] Jin Soung Yoo and Mark Bow. 2011. Mining top-k closed co-location patterns. In *Proceedings 2011 IEEE International Conference on Spatial Data Mining and Geographical Knowledge Services*. IEEE, 100–105.
- [27] Jin Soung Yoo and Shashi Shekhar. 2006. A joinless approach for mining spatial colocation patterns. *IEEE TKDE* 18, 10 (2006), 1323–1337.

A PARTICIPATION-RATION BOUND

LEMMA 2.1. *If a parent feature-type ($\widehat{f}$) has no instances i.e., $|\widehat{f}| = 0$, then its participation ratio is always bounded by the participation ratio of the most participant child and the least participant child in a colocation C of the form $\{ch(f), G\}$, where $G \wedge ch(f) = \emptyset$ and $G = \{g_1, \dots, g_n\}$.*

PROOF. Let $ch(\widehat{f}) = \{f_A, f_B, \dots, f_N\}$, f_I be the most participant child and f_J be the least participant child. Then,

$$pr(C, f_J) \leq pr(C, \widehat{f}) \leq pr(C, f_I).$$

Consider

$$pr(C, \widehat{f}) = \frac{\sum_{i=1}^N Count(C, f_i)}{\sum_{i=1}^N |f_i|}$$

Since f_I is the most participant child, we get

$$\begin{aligned} \frac{Count(C, f_i)}{|f_i|} &\leq \frac{Count(C, f_I)}{|f_I|} \\ \implies Count(C, f_i) &\leq \frac{|f_i| * Count(C, f_I)}{|f_I|} \\ \implies \frac{\sum_{i=1}^N Count(C, f_i)}{\sum_{i=1}^N |f_i|} &\leq \frac{\sum_{i=1}^N \frac{|f_i| * Count(C, f_I)}{|f_I|}}{\sum_{i=1}^N |f_i|} \\ \implies pr(C, \widehat{f}) &\leq \frac{\sum_{i=1}^N Count(C, f_I)}{\sum_{i=1}^N |f_I|} = pr(C, f_I). \end{aligned}$$

Similarly, the left inequality can be proved using the property of the least participant child. $\square$